

Software-Defined GPS Receiver and Position Accuracy Analysis Using RTL-SDR

Sarah Bashandy

Abstract—This project implements a software-defined GPS receiver using an RTL-SDR V4 dongle and an active antenna and evaluates the factors that most influence position accuracy in urban and suburban environments. The receiver captures raw IQ samples at the GPS L1 frequency (1575.42 MHz), performs satellite acquisition, tracks carrier and code phase, decodes navigation messages, and computes position solutions. Data was collected at seven distinct locations across the UCLA campus and in Tracy, California, varying satellite count, signal-to-noise ratio (SNR), and environmental conditions including sky obstruction, multipath reflections, and antenna ground-plane configuration. Results demonstrate that a functional GPS receiver can be constructed from an RTL-SDR, achieving horizontal accuracy within 5-10 meters when seven or more satellites with SNR above 25 dB-Hz are available in a suburban open-sky environment. However, the receiver is incredibly susceptible to environmental effects and is prone to error. Including weak satellites below approximately 25–35 dB-Hz actively degrades the position solution, and vertical accuracy consistently remains 1.5× to 3× worse than horizontal accuracy due to poor vertical dilution of precision (VDOP). Environmental factors, particularly tree canopy, building proximity, and partial sky visibility, dominate signal quality more than receiver altitude or multi-constellation augmentation.

Keywords—RTL-SDR, GPS, Software-defined radio, pseudorange, position accuracy, SNR, dilution of precision, GNSS, satellite acquisition, PLL/DLL tracking.

I. INTRODUCTION

A. Tech history

The Global Positioning System (GPS) was developed by the United States Department of Defense in 1973 as a satellite-based navigation system to provide worldwide, all-weather positioning capability. GPS operates by transmitting precise timing signals from satellites in medium Earth orbit (~20,200 km altitude); receivers on the ground measure the travel time of these signals and triangulate position from four or more simultaneous measurements [2].

Each GPS satellite transmits on the L1 carrier frequency (1575.42 MHz) using binary phase-shift keying (BPSK) modulation. The coarse/acquisition (C/A) code is a 1,023-chip pseudorandom noise (PRN) Gold code repeating every millisecond at 1.023 Mchip/s available for civilians to

decode navigation messages. A 50 bps navigation message is superimposed, carrying satellite ephemeris (orbital parameters), almanac data, clock corrections, and ionospheric model coefficients [1], [3]. To decode position from the raw radio signal, a receiver must: (1) acquire satellites by searching a two-dimensional space of Doppler frequency offsets and code phases; (2) track each satellite via phase-locked loops (PLL) and delay-locked loops (DLL); (3) decode the navigation message to extract transmission timestamps and position data; and (4) compute pseudoranges and solve for position and clock bias [2].

Software-defined radio (SDR) emerged in the 1990s as a paradigm for implementing radio functions in software rather than dedicated hardware. The RTL-SDR was released in 2010 and it's a general-purpose receiver covering 24 MHz to 1.7 GHz and a stable sampling rate of up to 3MSPS. The RTL-SDR Blog V4, released in 2023, included an integrated bias-tee and a TCXO, making GPS L1 reception feasible for under \$40 in total hardware cost [4]. Open-source GPS receiver projects including GNSS-SDR [5] and lots of MATLAB implementations [2] demonstrated the educational and research value of software-defined GPS reception, though most assume front-end hardware with higher sampling rates and better noise performance than the RTL-SDR provides.

B. External Impact

We use GPS every day to get where we are going. Most technology has GPS capabilities. Emergency services use GPS to help people in need and so do search and rescue teams. It's important to understand how GPS works and what negatively affects GPS signals and how to mitigate these effects.

With increasing drone usage in everyday life, cheap GPS modules are now something that everyday people are encountering. If someone sends out their drone and configures its GPS module to use any and all satellites it acquires no matter the SNR value it could potentially end up getting lost or it could bump into someone and hurt them.

Safety and welfare: The finding that adding weak satellites degrades rather than improves position accuracy and finding a rough estimate of that SNR for this specific receiver has direct safety implications. Low-cost GPS modules used in personal trackers, IoT devices, and consumer

drones may be defaulting to a "use all available satellites" strategy that is suboptimal in obstructed environments. This project's quantitative SNR thresholds (25–35 dB-Hz) provide actionable guidance for improving position reliability in consumer applications.

Economic: By demonstrating that a \$35 SDR dongle can achieve 5-meter positioning under favorable conditions, this project lowers the barrier to entry for GPS experimentation and education. Engineering students and hobbyists can now build and characterize a complete GPS receiver for a fraction of the cost of commercial evaluation kits, facilitating broader participation in GNSS research and development.

II. MOTIVATION

A. Tech Contribution

The project was also an opportunity for me to learn how GPS is implemented and it was fun. It helped me explore the limitation of the SDR dongle and different ways of running code on the RTL-SDR.

Most existing open source GPS receivers, like GNSS-SDR [5], are designed with much higher sampling rates in mind. GNSS-SDR is designed for front ends that have 4-16MSPS at the minimum. The RTL-SDR V4, with a maximum usable bandwidth of 2.046 MSPS and a higher noise figure functions at the edge of GPS feasibility [8]

The GPS receivers that are designed with RTL-SDR sampling rate limitations in mind are either very basic position solvers that have 1-5km position accuracy and are very rudimentary.

A common assumption in GPS receiver design is that more visible satellites always improve the position solution [9]. This project specifically tests and disproves this assumption for low-cost SDR hardware, demonstrating that including satellites below 25–35 dB-Hz C/N_0 actually worsens the fix. This finding has direct implications for receiver satellite selection algorithms: an SNR-quality-weighted strategy outperforms a simple count-maximizing strategy for RTL-SDR-class hardware.

Also, Multi GNSS solutions were compared with GPS-only solutions and the difference between the computed latitude and longitude were negligible however Galileo satellites improve the drifting height solution.

B. Projected External impact

Findings could potentially help students decide design parameters for their low cost GPS receivers and make more informed decisions for how they want to set up their receivers.

The project has educational value because it creates a clearer bridge between GNSS theory and hands-on implementation. Students learning about acquisition, tracking, navigation message decoding, and position estimation can use this kind of system to study real receiver tradeoffs on affordable hardware. In that sense, the project supports wider learning, experimentation, and innovation in GPS and GNSS technology.

III. APPROACH

A. Team Organization

I did everything by myself.

I did use existing open source code for some parts so credit to Chris Doble [10] whose code I based my code upon.

B. Plan and implementation

The initial plan was to test my hardware using open source code and then if it worked I would have confirmation that it was possible to build a gps receiver with my SDR and antenna and then write my own code for analysis. Unfortunately, I could not get any of the online GNSS-SDRs to work until week 3 because of the way the configurations were set up. I had to change a lot of the settings and make it less sensitive to noise and errors. For example, initially, they would discard a satellite if there was 3 errors in a PRN code and I made it less so that it would discard if there were 5 errors.

In the meantime, I tested my Antenna using a VNA and found that the internal bandpass filter was centered around the expected frequency and so in theory it should work fine. I started independently writing my own code based on online GPS receivers I found.

Eventually I could record IQ values and then extract navigation bits however I could not get my trigonometric solver to correctly compute position values until week so I relied on RTK_LIBPOST to do analysis on the data I processed.

Table 1 summarizes the planned milestones and actual progress:

TABLE I: PLAN VS. IMPLEMENTATION

Milestone	Intention	What Happened
Week 3	Acquire hardware; record IQ samples; implement acquisition, tracking, and position lock; begin satellite-count vs. accuracy analysis	Hardware acquired and IQ samples recorded. Acquisition (FFT correlation) worked. Tracking loops (PLL/DLL) failed to lock for two weeks due to coarse Doppler estimates and overly tight initial PLL bandwidth. Position lock achieved in Week 3 using open-source software and my code combined after widening pull-in bandwidth to 500 Hz and adjusting loop gains. Initial position data from Boelter Hall showed 100+ km errors with only 4–5 weak satellites.
Week 7	Collect multi-site environmental data; complete satellite count, SNR, and DOP analysis; compare GPS-only vs. multi-GNSS	Finished configuration settings (ms of acquisition, step size for FFT, etc.) in Tracy, CA as it was a more open environment with less interference. Collected excellent data in Tracy, CA showing 5 m accuracy with 7 satellites. Returned to UCLA and discovered navigation file mismatch causing solver failures; corrected.
Week 9	Collect more data and analyze position accuracy.	Collected data at 5 UCLA campus sites (March 3). Completed SNR threshold, DOP, and environmental factor analyses. Discovered intermittent LNA power issue causing SNR drops during recordings. Cut around existing data to remove bad SNR readings in analysis.

An example of how samples were obtained can be seen in [fig 1].



Fig 1: Computer set up on side walk to collect IQ samples

The RTL-SDR was tuned to 1.575 GHz with a bandwidth of 2.046MSPS (2.048MSPS would have worked better because) and placed on a toaster tray which acted as a ground plane. Using GQRX to record the sample, DC remove was turned on, ACG off and IQ correction on. The receiver was turned on for 5 minutes prior to recording samples so that temperature variation don't affect samples and then when recording started, time was recorded (The application I used to record IQ samples had an option for adding time stamps but I also manually wrote down the time because it often became an issue because it would not properly record it). During Quality analysis later on, I discovered that there were chunks of data where the average SNR drops from 50-60dB-Hz to 20-30dB-Hz as seen in [fig 2]. It's unlikely that the GPS frequencies experienced interferences so likely this was either a connectivity issue or a bad LNA.

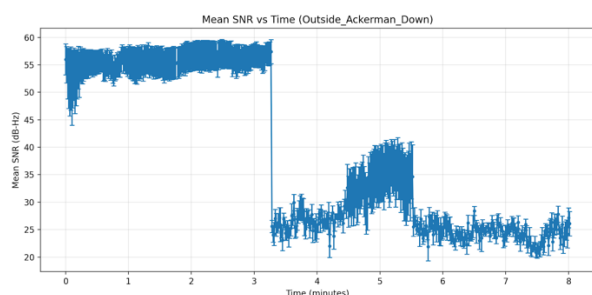


Fig 2: SNR vs time of sample.

The ground truth position was determined using the compass app on my iPhone (I placed it exactly where the antenna is placed before recording samples). This is not the correct ground truth but I figured it was accurate enough for the purposes of my analysis. Important to note that the compass app also struggles to lock height data although it is accurate to ± 10 meters.

My implementation was split up into 6 steps:

1. Record IQ samples
2. Perform Acquisition
3. Track satellites that meet minimum Acquisition threshold.
4. Compute Pseudoranges using navigation data.
5. Quality testing—discard anomalous results.
- 6a. Compute position using trigonometric solver using computer Pseudoranges.
- 6b. Compute position using RTK-lib post using computed pseudoranges and compare with trigonometric solver using.

Although the purpose of all this was initially to analyze position accuracy vs satellite count starting with the strongest satellites and adding the weaker ones, initial attempts to acquire more satellites led me to use satellites whose SNR was very low and weakly acquired satellites and so in the end I also analyzed the minimum SNR range where adding a satellite introduced a large amount of error.

C. Theory acquisition

The initial idea came from Dr. Roberts who suggested I build a GPS receiver from an SDR and after that I looked into different open source GNSS receivers.

I read through Borre et al.'s textbook on software-defined GPS receivers [2], which provided the signal processing pipeline from IQ samples through position computation and explained the mathematics behind it which helped me write the code.

The GPS Interface Control Document (IS-GPS-200) [1] and the Wikipedia page for GPS defined the signal structure, PRN code generation, and navigation message format.

Several open-source GNSS GitHub repositories were studied to understand practical implementation details, particularly regarding tracking loop bandwidth selection and pull-in strategies. Chris Doble[10] and Michael Van Biezen's [12] YouTube videos on GPS receivers also helped. Chris Doble's code was also what I started with (although it did not work at all I had to make a lot of edits to the configuration files).

RTKLIB documentation [6] provided the post-processing methodology used for solution verification. Iterative experimentation, recording samples, processing them, analyzing failures, and adjusting parameters, was essential for bridging the gap between textbook theory and practical RTL-SDR implementation

The RTL-SDR website[11] has an article about implementing a GPS receiver using GNSS-Lib and RTKLIB however that application is a little outdated and when I tried running it on the V4 dongle I found that I had to edit some of the configurations to get performance out of it because it had been configured for the V3. However, once I did get it to work with my set up I found that both those solvers are incredibly accurate (Horizontally speaking they were within 5-10 meters of truth, vertically they were just as bad as my final code).

D. Theory

In order to compute your position using GPS satellites using the IQ samples that were recorded you first have to decode the navigation message and compute your pseudorange. A pseudorange represents the distance between the GPS satellite at the time of the emission of the signal and the receiver antenna measured at the time of reception[13]. There are 5 main steps to do this:

1. Acquisition

During Acquisition, you are trying to determine which satellites are visible and if they are visible, estimate their (1) PRN code phase, which identifies where in the satellite's 1,023-chip pseudorandom code the received signal aligns, and (2) the Doppler frequency shift, which is the offset of the carrier frequency caused by the relative motion between the satellite and the receiver plus any error in the RTL-SDR's local oscillator.

Each GPS satellite transmits a unique C/A code—a 1,023-chip Gold code generated by two 10-stage linear feedback shift registers. Both registers are initialized to all ones and clocked at 1.023 MHz, repeating every 1 millisecond. Each satellite's code is formed by XORing G1 with a satellite-specific delayed version of G2 [1]. The receiver generates a local replica of each satellite's code and correlates it against the received signal to detect the satellite.

Acquisition searches a two-dimensional grid across all possible code phases (0–1,022 chips) and Doppler frequency offsets. The search was implemented using FFT-based parallel code-phase correlation: for each candidate Doppler bin, the received signal is frequency-shifted to remove the trial Doppler, then circularly correlated with the local PRN code replica.

The result of this search for each satellite is a 2D correlation surface: code offset on one axis, Doppler frequency on the other, and correlation strength on the vertical axis as seen in figure 3. A successfully detected satellite produces a sharp, clearly defined peak rising above the noise floor. A satellite that is not visible or is too weak shows only a flat noise surface with no discernible peak. In order to move on to the tracking phase we have to put a minimum correlation strength requirement. Because the receiver is so sensitive to noise, I had to significantly lower this number to 1.8 and most online GNSS receivers use values of 3+ however most of the satellites I acquired had anything close to that value.

In order to get higher correlation values we can integrate over more ms of data since the PRN codes repeat every 1ms but the navigation bit changes every 20ms. This can help us detect weaker satellites. That means we have to be strategic with the timeframe we choose. If we choose to integrate over a number greater than 20ms of data then the navigation bit might flip and correlation values will drop significantly. In theory, a good number is 10-20ms of data however I found on my end that <10ms of data would give me higher numerical values but that did not correspond to better

SNR values or more coherent data and in fact a lot of those satellites were quite unusable. In the end I went with 10ms of data just based off of what I read and what worked for me.

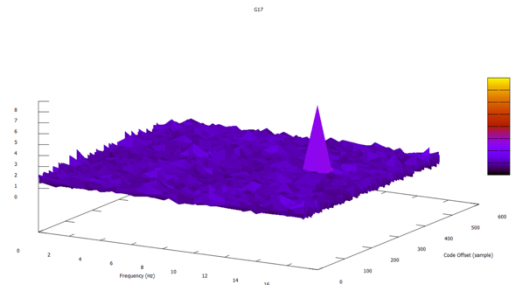


Fig 3a: Example of good strong signal acquired

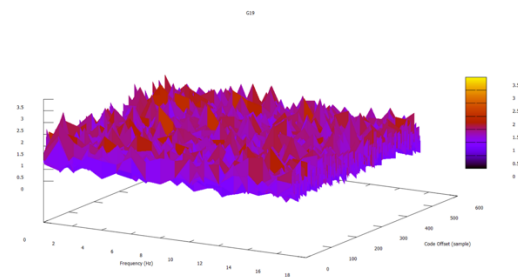


Fig 3b: Example of satellite that would inaccurately move on to the tracking stage (peak >1.8).

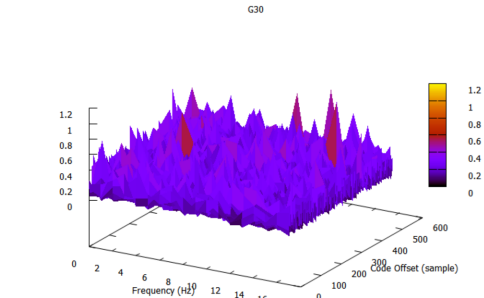


Fig 3c: Example of satellite that was not acquired (strength <1.2)

The Doppler search range = ± 10 kHz (accounts for maximum satellite Doppler of ± 5 kHz plus RTL-SDR oscillator error). The Doppler step size = 500 Hz (41 bins); Integration time = 1 ms per block (one C/A code period), extended to 10 ms coherent integration for weak signals (basically all). Detection threshold = normalized correlation metric > 1.5-1.8 depending on location (peak divided by noise floor mean).

So from each satellite you should get: code phase, doppler frequency estimate, and a normalized correlation metric. Early indoor tests (ground floor, windows open) detected 6–7 satellites during acquisition but many had very weak correlation metrics (below 1.5), making them unusable for subsequent tracking. Moving to a rooftop or adding a metal ground plane beneath the antenna significantly improved correlation

metrics by reducing below-antenna multipath and improving the antenna's upward directivity. The ground plane was the single most effective hardware change.

2. Tracking

Acquisition provides a rough estimate of each satellite's code phase and Doppler, accurate to ± 0.5 chips and ± 500 Hz respectively. In order to properly decode the navigation message, these values have to be way more accurate. This is the trickiest part of the whole process.

Tracking refines these estimates continuously using two closed-loop feedback systems that operate in parallel: a phase-locked loop (PLL) for the carrier frequency/phase, and a delay-locked loop (DLL) for the code phase. Tracking must maintain lock while the satellite moves across the sky, the receiver oscillator drifts, and environmental conditions change. You need to have a lock on the satellites for at least 6 seconds.

The PLL tracks the carrier frequency and phase using a second-order Costas loop, which is insensitive to the 180° phase ambiguity from BPSK data modulation. Every 1 ms (one code period), the receiver multiplies the incoming signal by local carrier and code replicas to produce in-phase (I) and quadrature (Q) correlator outputs.

So the PLL has an Integration interval of 1 ms, Pull-in bandwidth = 500 Hz, Steady-state tracking bandwidth = 10–20 Hz, Lock detection = prompt correlator magnitude above threshold for 5+ consecutive intervals.

This was the most difficult obstacle in the entire project. Initially, the PLL bandwidth was set to 10–20 Hz following standard textbook recommendations for GPS tracking. However, the acquisition Doppler estimates are only accurate to approximately ± 500 Hz—which is far outside the pull-in range of a 10–20 Hz PLL. The result was that the loop never achieved lock: the Doppler estimate diverged instead of converging, and the prompt correlator magnitude (magP) stayed near zero. This failure mode was not immediately obvious because the code appeared to be running correctly—the issue was a parameter mismatch, not a bug.

After reading through Borre et al. [2], I found a section that advised a two-phase tracking strategy: start with a wide PLL bandwidth of 500 Hz for initial pull-in (allowing the loop to capture the signal despite the coarse Doppler estimate), then automatically narrow to 10–20 Hz once stable lock is detected. This was implemented by adjusting the proportional (kp) and integral (ki) values. Once implemented, tracking lock was achieved within 30 seconds on some satellites with C/N_0 above 18 dB-Hz (typically 25+ though).

The DLL had to find whether or not the doppler shift caused the signal to be early, prompt or late as seen in figure 4. The chosen parameters were Correlator spacing = ± 0.5 chips (early-minus-late), DLL bandwidth = 2 Hz. The DLL was much less problematic than the PLL.

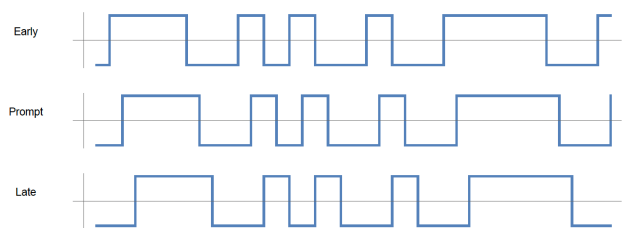


Fig 4: Doppler shifted signal

3. Navigation decoding

Once a satellite is being tracked, the navigation message embedded in the signal must be decoded. This message contains the satellite's transmission timestamp (Time of Week), which is essential for computing the pseudorange.

- Each 1 ms prompt correlator output from the tracking loop constitutes one pseudosymbol. The navigation data is modulated at 50 bps, meaning each data bit spans exactly 20 ms = 20 pseudosymbols.
- The receiver does not know which of the 20 possible pseudosymbol positions marks the start of a data bit. To find this boundary, the variance of summed blocks of 20 pseudosymbols is computed at each of the 20 possible offsets. The offset that produces maximum variance (sharpest transitions between +1 and -1 bits) identifies the correct bit boundary. Once found, every 20 consecutive pseudosymbols are summed and their sign determines the data bit value (+1 or -1).
- Search for the preamble: The navigation message is organized into subframes of 300 bits each, transmitted over 6 seconds. Each subframe begins with an 8-bit preamble pattern: 10001011. The decoded bits are searched for this pattern (and its BPSK-inverted complement 01110100) to establish subframe alignment [1]. In my receiver as long as there were less than 3 errors, I counted the preamble as found, obviously this introduces a lot errors but I found that it was the best possible solution to get access to enough satellites.
- Extract time of week from the bits[1].
- Compute Psuedorange:
 - o $T = t_{recieved} - t_{sent}$
 - o $Psuedorange(d) = cT$

You have time sent from the navigation message and time recieved from your computer and therefore you can compute pesudorange (d).

4. Position fix

With pseudorange measurements from four or more satellites, the receiver solves for its position and clock bias. The system has four unknowns—three position coordinates (X, Y, Z in the Earth-Centered Earth-Fixed coordinate frame) and the receiver clock error—requiring a minimum of four pseudorange equations. The solution is computed via iterative weighted least squares: an initial position guess (typically the center of the Earth or a

previous fix but I just hard coded LA location so that computation didn't take forever) is refined over several iterations until the corrections converge below a threshold [2].

The custom Python solver produced horizontal errors of 20–40 meters at campus sites, while RTKLIB's solution using the same observation data showed similar error magnitudes, confirming that the dominant error source was the input data quality (weak signals, ionospheric bias) rather than the solver implementation. A consistent ~6–9 km systematic bias was present in all UCLA solutions, attributable to uncorrected ionospheric delay inherent to single-frequency L1 reception. This bias does not affect relative comparisons (e.g., how error changes when adding or removing satellites from the same dataset).

E. overall setup

To replicate this GPS receiver system, the following procedure should be followed:

Equipment required: RTL-SDR Blog V4 dongle; active GPS L1 patch antenna with SMA connector; metal ground plane (any flat conductive surface $\geq 20 \text{ cm} \times 20 \text{ cm}$); laptop with GQRX installed; Python 3.x with NumPy, SciPy, and Matplotlib.

Step 1 – Hardware assembly: Connect the active antenna to the RTL-SDR's SMA input. Place the metal ground plane on a flat surface and set the antenna on top of it, face up toward the sky. Connect the RTL-SDR to the laptop via USB.

Step 2 – Site selection: Choose a location with maximum sky visibility. Avoid standing under trees (the single most destructive factor). Maintain at least 50 steps of distance from tall buildings if possible. Elevation above surrounding structures improves reception but is less important than sky clearance.

Step 3 – Recording: Launch GQRX, set center frequency to 1575.42 MHz, sample rate to 2.046 MSPS, and enable the bias-tee. Begin recording raw IQ data. Record for a minimum of 4 minutes to allow sufficient navigation message decoding time (a complete navigation message frame takes 12.5 minutes, but individual subframes containing TOW data arrive every 6 seconds). Note the exact start time (UTC) and geographic location (from phone GPS or survey).

Step 4 – Navigation file (optional): Download the broadcast navigation file for the recording date from NASA CDDIS (<ftp://cddis.nasa.gov/gnss/data/daily/>). Ensure the file date matches the recording date exactly; using the wrong navigation file was a significant source of solver failures in this project. This is probably more accurate than the navigation file and so can also be used instead.

Step 5 – Processing: Run the IQ file through the acquisition module to identify visible satellites and their code phase/Doppler estimates. Run the tracking module on each detected satellite to refine parameters and decode navigation

bits. Run the pseudorange module to compute range measurements. Run the position solver with the pseudoranges and satellite positions computed from the broadcast ephemeris. Optionally, format the observations as RINEX and process through RTKLIB for independent verification.

IV. RESULTS

A. Description of what went well

When I was in Tracy, CA (suburban area), I managed to obtain the results in figure 5 from any large building.

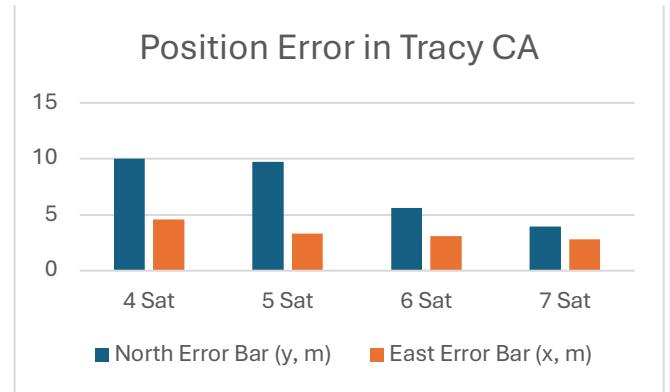


Fig 5: ANALYSIS OF POSITION DATA IN OPEN FIELD

These results were obtained using RTKLIB post. I solved for pseudoranges and then created 4 separate RINEX 3.02 observation files, containing the data for 4,5,6,7 satellites per epoch (1s). The computed position latitude and longitude values were then averaged with the exception of anomalies.

My least squared solver was within $\pm 10m$ of the results found. Even with RTKLIB post the error bars were extremely large (over 1km) so for ease of readability, they were not included in this figure. Vertical error bars were also not included in this figure because the elevation estimate never stopped drifting. For example, it would estimate that in one second I was at -300000m and one second later it would estimate that I was at 13m. For horizontal estimates, I could average out the estimates over a certain time frame but for elevation it was just too uncertain for me to do that.

The FFT-based acquisition engine reliably detected satellites with C/N₀ above approximately 15 dB-Hz, with processing time of approximately 2–3 seconds per satellite on a standard laptop. In the best environment tested (Tracy, CA, open sky), 9 satellites were detected with correlation metrics ranging from 1.5 to 3.5. In the worst environments (trees + building), only 1–2 satellites were detectable and none were usable for positioning.

In table 1 you can see the effect of different environments on the average SNR of the top satellites and the effects of these environments. In table 1, a visible satellite was defined as a satellite with a peak correlation value of 1.8 or larger and a useable satellite was one where I could read the navigation message.

Condition	Visible Sats	Usable Sats	Top-2 Avg SNR (dB-Hz)	Std Dev
Clear sky, >50 steps from building	8	6	59.5	7.1
Clear sky, <50 steps from building	7	5	52.4	12.6
Trees overhead, away from building	4	1	31.7	12.0
2 umbrellas overhead, away from bldg	1	1	22.6	2.7
Trees overhead + near building	2	0	0	0

TABLE 1: DIFFERENT ENVIRONMENTS TESTED

Using the same recorded data set, I tested multi-GNSS analysis vs GPS analysis (Using the strongest 4 satellites but including Galileo and GLONASS satellites). At 4 satellites, multi-GNSS produced dramatically worse results: 1.3 km horizontal drift and a 4.3 km vertical collapse, compared to GPS-only's 10 m horizontal error. At 6+ satellites, both approaches converged to similar accuracy. The multi-GNSS 4-satellite failure is attributable to the inclusion of non-GPS satellites with lower signal quality from the RTL-SDR's perspective (the receiver's configuration was optimized for GPS L1 C/A).

Vertical uncertainty remained high ± 9 m) regardless of satellite count or constellation choice, consistent with the inherently weak vertical geometric constraint from satellites above the horizon.

UCLA Campus Multi-Site Trials (March 3, 2026). Five sites across campus at elevations from 400 to 460 feet were selected to represent typical campus environments near buildings but with partial sky visibility. All recordings were made on the same evening within a 200-minute window to minimize satellite geometry changes between sites. Table 2a and 2b presents position errors computed for each site at varying satellite counts computed by RTKLIB and my solver respectively.

TABLE 2A: POSITION ERROR BY SITE AND SATELLITE COUNT (RTKLIB)

Site (Elevation)	n_sat	Horiz. Error (m)	Vert. Error (m)
Ackerman Down (400 ft)	4	10.3	-18.3
Ackerman Down (400 ft)	5	28.0	-26.9
Ackerman Up (420 ft)	4	26.5	-49.0
Ackerman Up (420 ft)	5	32.7	-60.0
Ackerman Up (420 ft)	6	28.3	-48.1
Ackerman Up (420 ft)	7	30.2	-14.8
Kerckhoff Patio (430 ft)	4	24.2	-27.5
Kerckhoff Patio (430 ft)	5	17.0	-41.2
Math Sciences (440 ft)	4	8.3	-80.4
Math Sciences (440 ft)	5	37.1	-53.8
Outside Powell (460 ft)	4	17.4	-60.0
Outside Powell (460 ft)	5	15.4	-31.4

TABLE 2B: POSITION ERROR BY SITE AND SATELLITE COUNT (MY SOLVER)

Site (Elevation)	n_sat	Horiz. Error (m)	Vert. Error (m)	Hor_std	Vert_std
Ackerman Down (400 ft)	4	67,746	-100,500	835.722	12234.72
Ackerman Down (400 ft)	5	64,112	-96,228.8	657.710	283.81
Ackerman Down (400 ft)	6	64,110	-92,012.3	372.722	923.81
Ackerman Up (420 ft)	4	1,550,773	-300	143.6743	84.21223
Ackerman Up (420 ft)	5	630,230	-3300	84.4352	87.33
Ackerman Up (420 ft)	6	123,337	-284	82.338	99.229
Ackerman Up (420 ft)	7	80,000	-14.8	54.827	155.838
Kerckhoff Patio (430 ft)	4	66.0	-41	22.384	41.33
Kerckhoff Patio (430 ft)	5	23	-45	13.39	22.65
Math Sciences (440 ft)	4	29.5	-80.4	8.1	77.54
Math Sciences (440 ft)	5	24.5	-60.2	25	105.3
Math Sciences (440 ft)	6	24.5	-55.8	18.2	22.7212381
Outside Powell (460 ft)	4	18.0	-60.0	3.229	24.53561333
Outside Powell (460 ft)	5	17.5	-13.8	4.6507	13.4213

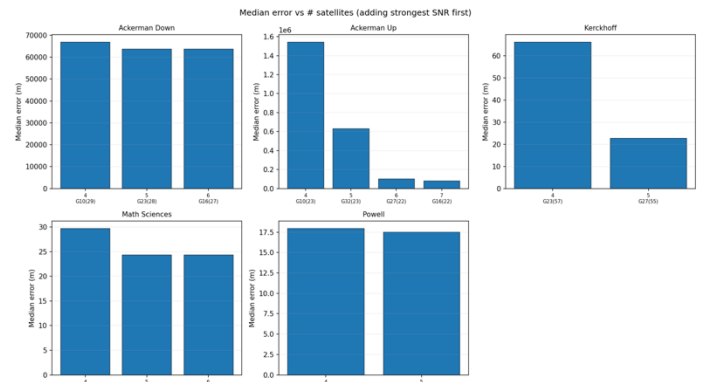


Fig 6: POSITION ERROR BY SITE AND SATELLITE COUNT (MY SOLVER) BAR CHARTS

You can clearly see that there are huge inaccuracies with the Ackerman sites position estimates. Once again, error bars were not included because they were very large and would have made it difficult to read the graphs.

I had done quality testing and thrown out anomalous data points but unfortunately the Ackerman sites had lower SNR values than the rest of the sites as seen in [fig 7].

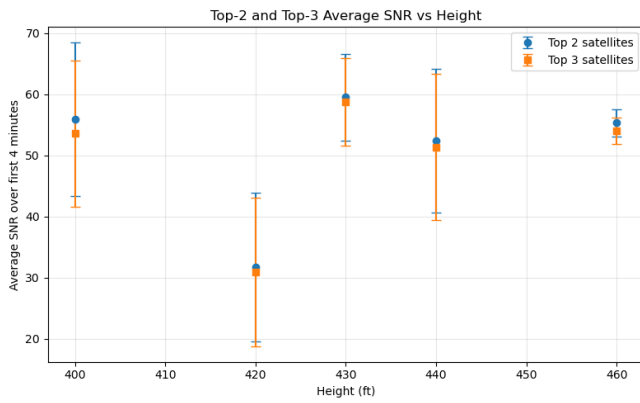


Fig 7: SNR VS HEIGHT (REPRESENTING POSITION HERE BECAUSE THE HEIGHT DIFFERENCE IS SO NEGLIGIBLE).

The difference between the accuracy of the RTKLIB solver and my solver is that RTKLIB has a compensation factor for ionospheric interference while mine is a very basic trigonometric solver.

Furthermore, I finally went back to Boelter hall roof to try and get a position lock there and although I could not get a position lock, I managed to plot the elevation of the satellites I could see vs SNR and I noticed that the satellites I saw (4), were all within the same 3-degree window (fig 8) which suggests that there is something blocking the signal.

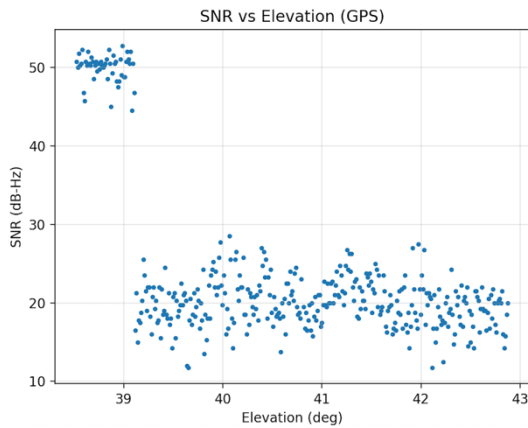


Fig 8: SNR VS ELEVATION OF SATELLITE OVER A 20 MINUTE WINDOW

SNR Threshold Analysis.

To quantify the SNR boundary below which including a satellite degrades accuracy, per-epoch horizontal errors were binned by the minimum satellite C/N_0 in each epoch's solution. Epochs where the weakest included satellite had C/N_0 below 30 dB-Hz produced zero fixes with error under 50 m. Above 35 dB-Hz, approximately 65% of epochs achieved sub-50 m error, and above 45 dB-Hz, over 85% did. Table 3 summarizes the per-site threshold behavior.

TABLE 3: SNR THRESHOLD ANALYSIS BY SITE

Site	Available Sats (SNR Range)	Best # Sats	SNR Where Degradation Began
Ackerman Down	6 (27–48 dB)	4	28 dB-Hz
Ackerman Up	7 (22–44 dB)	4	23 dB-Hz
Kerckhoff Patio	5 (55–60 dB)	5 (all helped)	N/A — all strong
Math Sciences	6 (32–58 dB)	4	32 dB-Hz
Outside Powell	5 (48–54 dB)	5 (all helped)	N/A — all strong

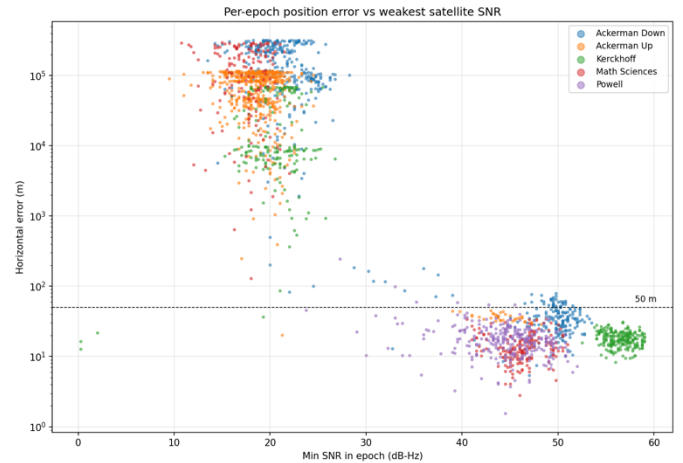


FIG 9a: MINIMUM SNR USED VS HORIZONTAL ERROR per epoch.

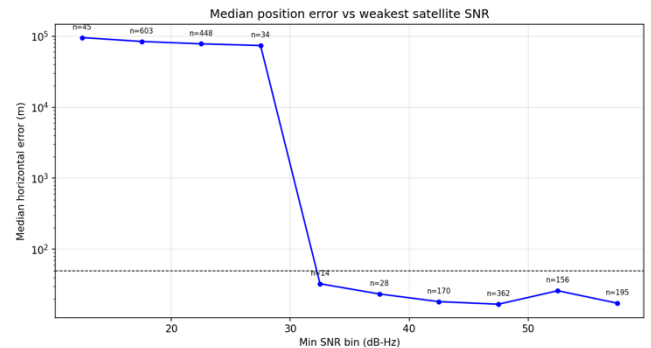


FIG 9b: MINIMUM SNR USED VS MEDIAN HORIZONTAL ERROR BUT BINNED.

The pattern is clear: at sites with uniformly strong signals (Kerckhoff, Powell), every satellite helped. At sites with a mix of strong and weak signals (Ackerman, Math Sciences), the optimal strategy was to use only the 4 strongest satellites and exclude anything below approximately 28–32 dB-Hz. A satellite selection algorithm based on SNR thresholding would outperform the default strategy of including all tracked satellites.

Dilution of precision (DOP) values were computed for the all values to determine if there was anything major affecting the data. Across all sites, VDOP exceeded HDOP by a factor of $1.5\times$ to $2.9\times$, confirming the theoretical expectation that vertical accuracy is inherently geometry-limited. Math Sciences exhibited the highest overall DOP (GDOP = 7.2, VDOP = 5.6) due to narrow sky visibility from surrounding buildings, which constrained the azimuthal distribution of

satellites. Kerckhoff Patio showed elevated VDOP (4.1) despite having the strongest individual signals, demonstrating that signal quality and satellite geometry are partially independent factors. A site can have excellent SNR but poor geometry, or vice versa.

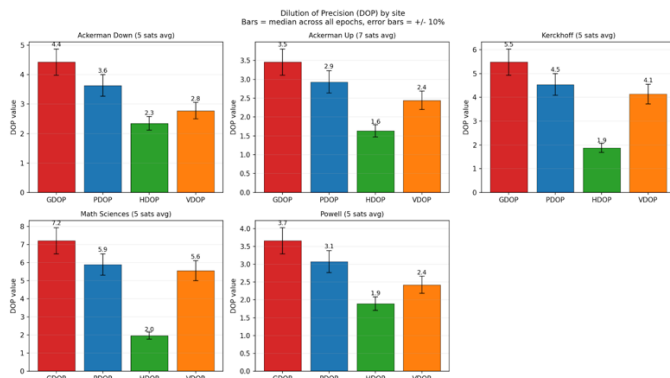


FIG 10: DILUTION OF PRECISION VALUES

B. Discussion

So, overall an RTL-SDR setup can produce usable GPS position solutions in favorable environments. In Tracy, CA with open sky and 7 satellites above 25 dB-Hz, horizontal accuracy of approximately 5 meters was achieved. However it is very sensitive to noise, even in favorable conditions.

Satellite signal quality (C/N_0) is a more important predictor of position accuracy than satellite count for RTL-SDR-class hardware. Including satellites below 25–35 dB-Hz actively worsens the position solution by introducing pseudorange measurement noise that outweighs the geometric benefit of an additional satellite.

Environmental factors, particularly overhead obstructions (tree leaves, buildings, etc.) and the multipath reflections, dominate the position error budget far more than hardware configuration, receiver altitude, or multi-constellation augmentation. Moving 50 steps away from a building improved average SNR by 7 dB-Hz and reduced signal variability by 44%. Tree canopy reduced usable satellites by 83%. These environmental effects dwarf the differences between GPS-only and multi-GNSS configurations.

Dual-frequency receivers eliminate ionospheric effects by exploiting the ionosphere's dispersive properties at different frequencies, but the RTL-SDR is limited to L1-only. Implementing the Klobuchar broadcast ionospheric model (whose parameters are transmitted in the navigation message) could reduce this bias by approximately 50%, but was not implemented in this project due to time constraints. For the relative analysis (how does error change when satellites are added or removed from the same recording), this systematic bias cancels and the conclusions remain valid.

Another thing that could have been done is using the L1C code in addition to the C/A code but alas no time.

Acquiring satellites is easy, but successfully tracking them and configuring the loop variables for your environment so that you get a favorable result is incredibly time consuming. It

took me 5+ weeks to get the tracking loops to work and it was not intuitive at all or obvious. Even doing what other open source GPS receivers had done did not work and I believe it's very environmentally sensitive.

Several limitations constrain the generalizability of these results. Phone-derived coordinates were used as ground truth rather than surveyed reference points, introducing up to ~5 meters of reference uncertainty. Single-frequency operation prevents ionospheric correction. Each campus site was measured in only one recording session on one evening, so the results may be influenced by the specific satellite geometry at that time rather than representing average performance. The intermittent LNA power issue may have degraded data quality at some sites. Future improvements should include: implementing Klobuchar ionospheric corrections using broadcast model parameters; using a powered external bias-tee for stable LNA operation; conducting repeated measurements across multiple days and times to average over satellite geometry variation; comparing against a dual-frequency survey-grade receiver for proper ground truth; and extending the analysis to additional campus environments including indoor-outdoor transitions.

REFERENCES

- [1] GPS Directorate, "IS-GPS-200: Interface Specification GPS Space Segment/Navigation User Segment Interfaces," U.S. Space Force, 2022.
- [2] K. Borre, D. M. Akos, N. Bertelsen, P. Rinder, and S. H. Jensen, A Software-Defined GPS and Galileo Receiver: A Single-Frequency Approach. Boston, MA: Birkhäuser, 2007.
- [3] E. D. Kaplan and C. J. Hegarty, Understanding GPS/GNSS: Principles and Applications, 3rd ed. Norwood, MA: Artech House, 2017.
- [4] RTL-SDR Blog, "RTL-SDR Blog V4 Datasheet and User Guide," 2023. [Online]. Available: <https://www.rtl-sdr.com/V4/>
- [5] C. Fernández-Prades, J. Arribas, and P. Closas, "GNSS-SDR: An open-source tool for researchers and developers," in Proc. ION GNSS+, 2011, pp. 780–794.
- [6] T. Takasu and A. Yasuda, "Development of the low-cost RTK-GPS receiver with an open source program package RTKLIB," in Int. Symp. GPS/GNSS, Jeju, Korea, 2009.
- [7] W. Gurtner and L. Estey, "RINEX: The Receiver Independent Exchange Format Version 3.02," International GNSS Service, 2013.
- [8] [1] C. D. Johnson, "gypsum," GitHub repository. [Online]. Available: <https://github.com/codyd51/gypsum/tree/release/gypsum>. Accessed: Mar. 21, 2026.
- [9] Inside GNSS, "Would you prefer to have more signals or more satellites?," Inside GNSS, Apr. 1, 2017. [Online]. Available: <https://insidengss.com/would-you-prefer-to-have-more-signals-or-more-satellites/>. [Accessed: Mar. 21, 2026].
- [10] C. Doble, "gps-receiver/gpsreceiver," GitHub repository. [Online]. Available: <https://github.com/chrisdoble/gps-receiver/tree/master/gpsreceiver>. [Accessed: Mar. 21, 2026].
- [11] "RTL-SDR Tutorial: GPS Decoding and Plotting," RTL-SDR.com, Feb. 28, 2017. [Online]. Available: <https://www.rtl-sdr.com/rtl-sdr-tutorial-gps-decoding-plotting/>. [Accessed: Mar. 12, 2026].
- [12] Michel van Biezen, "Michel van Biezen," YouTube channel. [Online]. Available: <https://www.youtube.com/@MichelvanBiezen>. [Accessed: Mar. 21, 2026].
- [13] "Pseudorange," ScienceDirect Topics. [Online]. Available: <https://www.sciencedirect.com/topics/engineering/pseudorange>. [Accessed: Mar. 21, 2026].